

Desenvolvimento para Dispositivos Móveis

Widgets Básicos, Layout e
Statefull Widgets

Universidade Federal do Rio Grande do Norte

Introdução

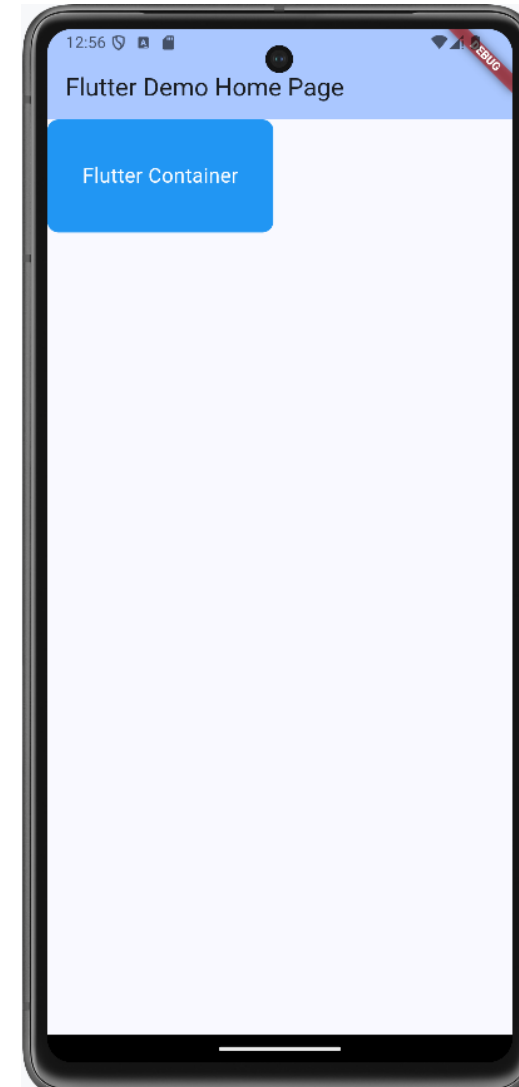
- Entender os layouts **permite que a criação de interfaces fluidas e responsivas**, otimizando a experiência do usuário.
- Vamos explorar como **organizar e estruturar** layouts em Flutter utilizando widgets fundamentais.
 - Container, Padding, Column, Row, Stack, Alignment, Expanded, Flexible, SizedBox.

Container

- O Container é um dos widgets mais flexíveis no Flutter.
- Serve como um "caixa" que pode **conter outros widgets** e **oferece** opções de personalização como **bordas, margens, tamanhos e alinhamento**.

Container

```
Container(  
  width: 200,  
  height: 100,  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Center(  
    child: Text(  
      'Flutter Container',  
      style: TextStyle(color: Colors.white, fontSize: 18),  
    ),  
  ),  
)
```

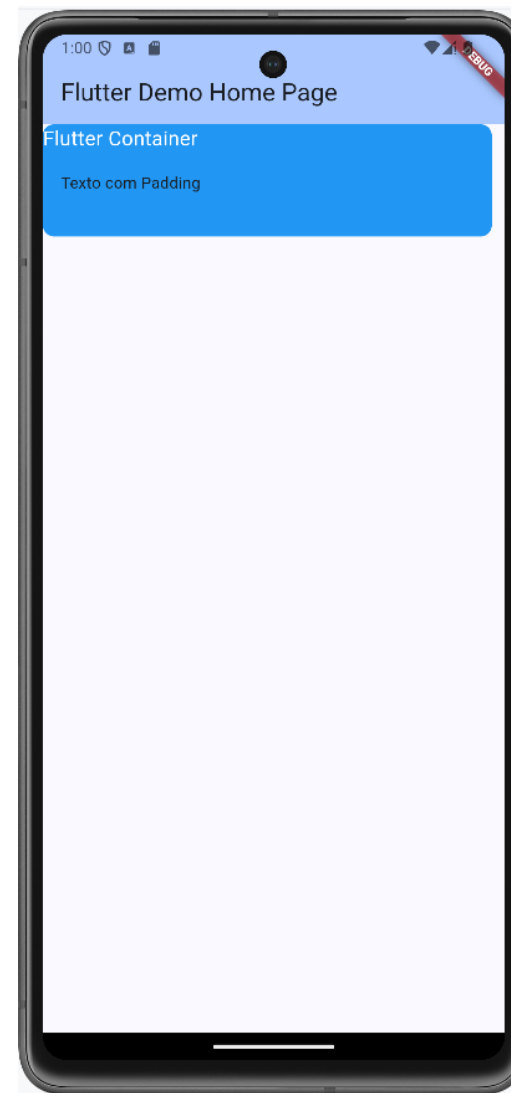


Padding

- O Padding adiciona espaço ao redor de seu filho, criando um espaço interno entre o conteúdo e o limite do widget pai.

Padding

```
Container(  
  width: 400,  
  height: 100,  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      Text(  
        'Flutter Container',  
        style: TextStyle(color: Colors.white, fontSize: 18),  
      ),  
      Padding(  
        padding: const EdgeInsets.all(16.0),  
        child: Text('Texto com Padding'),  
      ),  
    ],  
  ),  
)
```

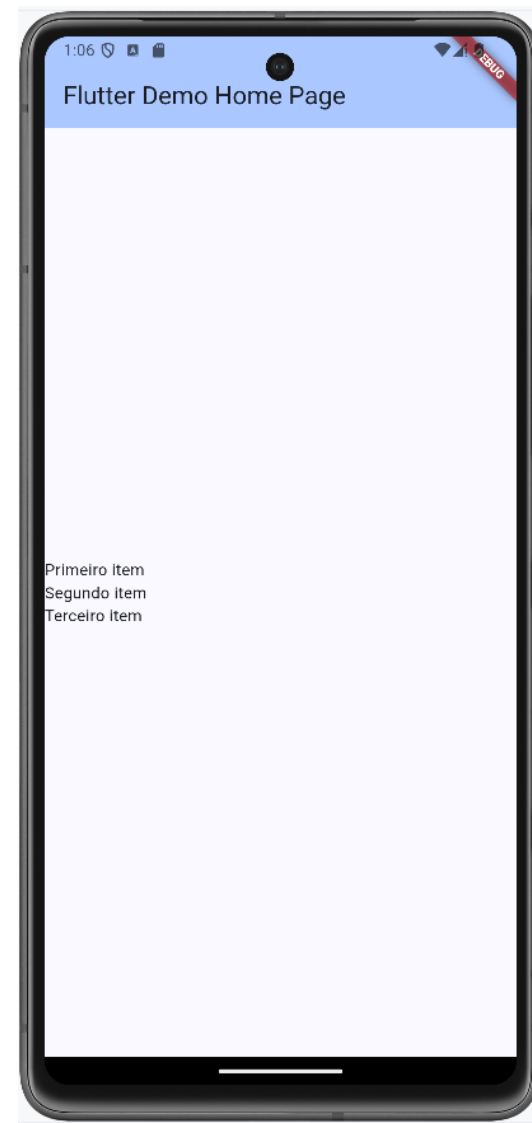


Column

- O Column organiza os widgets filhos **verticalmente**, de cima para baixo.
- Column pode ser usado para empilhar elementos como botões, imagens e textos de maneira organizada.

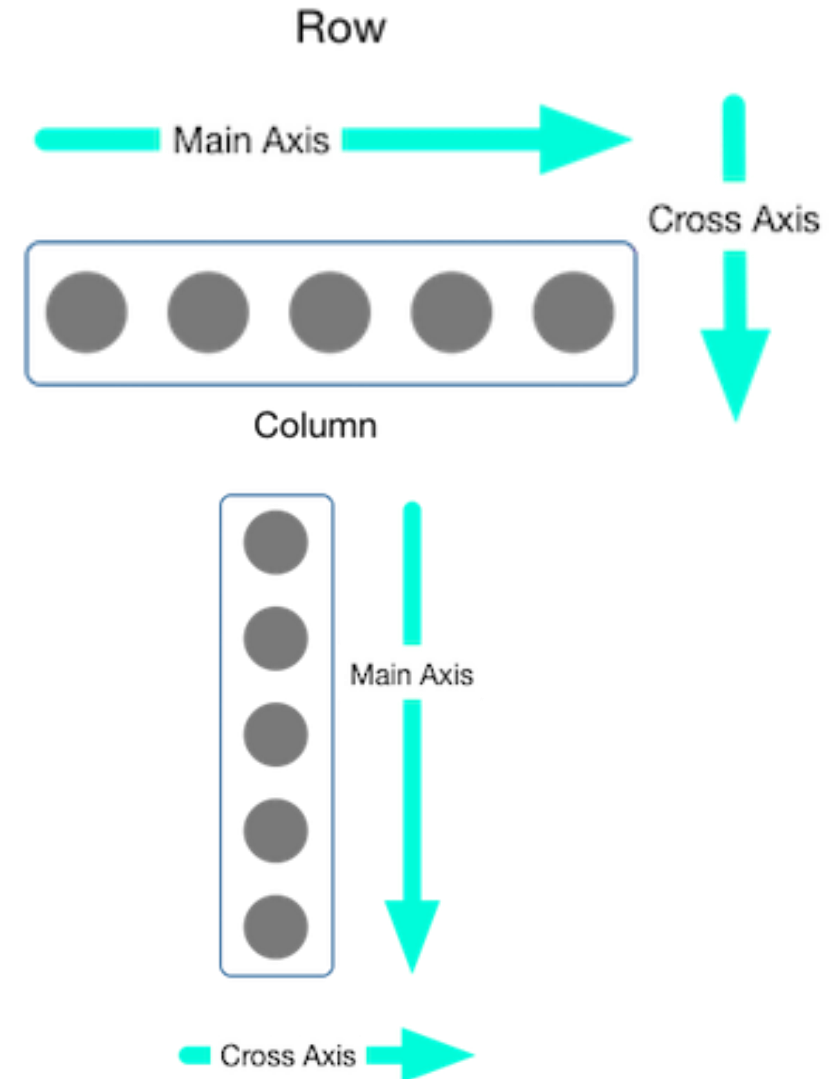
Column

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: [  
    Text('Primeiro item'),  
    Text('Segundo item'),  
    Text('Terceiro item'),  
  ],  
)
```



mainAxisAlignment e crossAxisAlignment

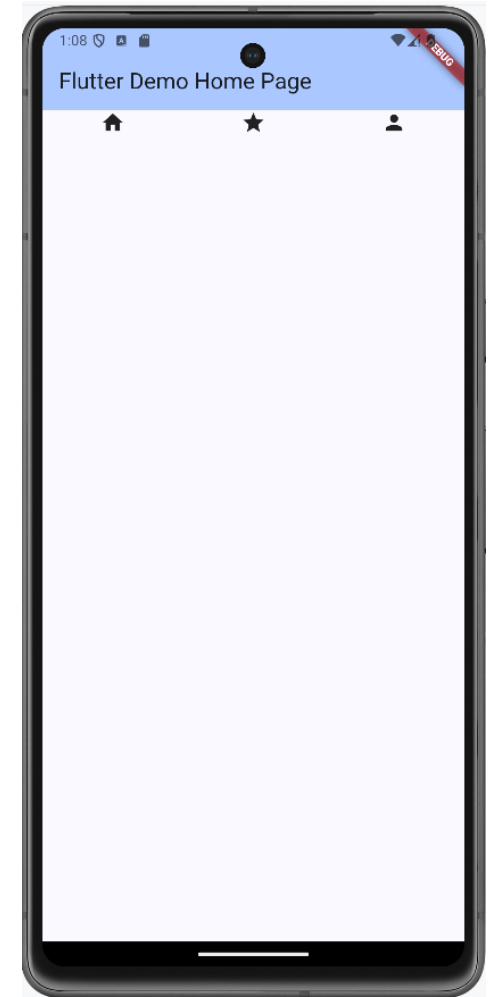
- É possível controlar como uma Row ou Column alinham seus filhos usando as propriedades mainAxisAlignment e crossAxisAlignment.
- Para uma Row, o eixo principal corre horizontalmente e o eixo transversal corre verticalmente.
- Para uma Column, o eixo principal corre verticalmente e o eixo transversal corre horizontalmente.



Row

- O Row organiza os widgets horizontalmente, de forma similar ao Column, mas na direção horizontal.

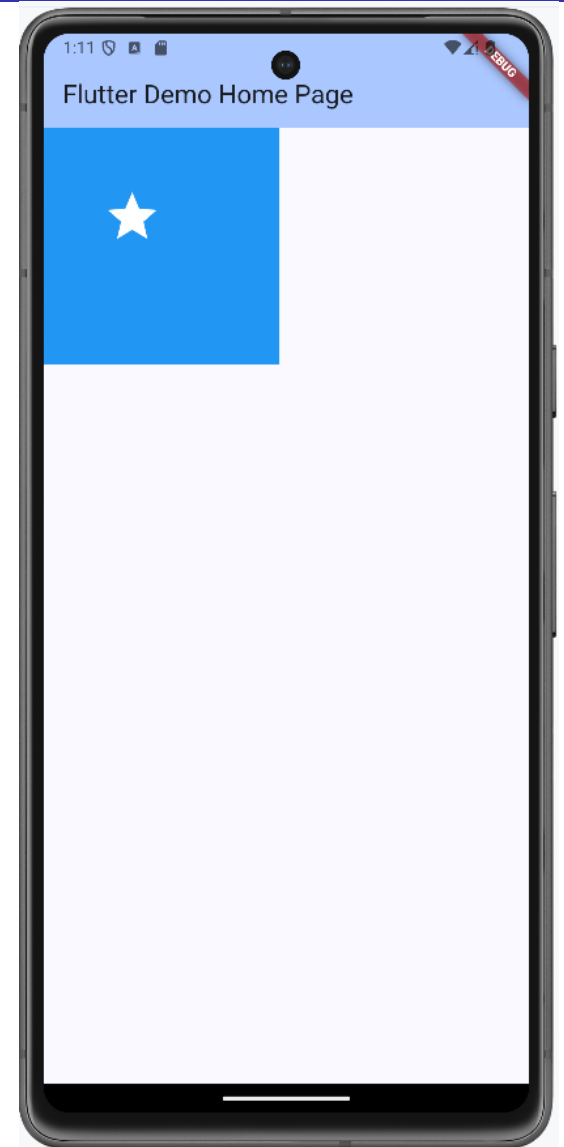
```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: [  
    Icon(Icons.home),  
    Icon(Icons.star),  
    Icon(Icons.person),  
  ],  
)
```



Stack

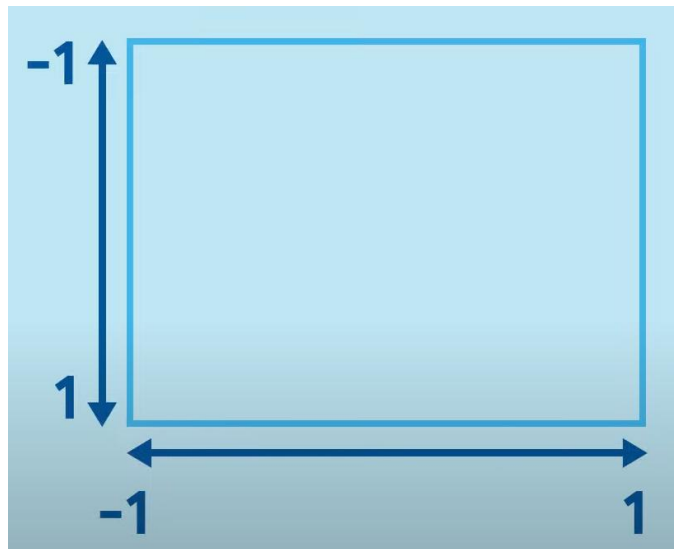
- O Stack permite sobrepor widgets uns sobre os outros.
- Útil para criar designs complexos como botões flutuantes. imagens sobrepostas. etc.

```
Stack(  
  children: [  
    Container(width: 200, height: 200, color: Colors.blue),  
    Positioned(  
      top: 50,  
      left: 50,  
      child: Icon(Icons.star, color: Colors.white, size: 50),  
    ),  
  ],  
)
```



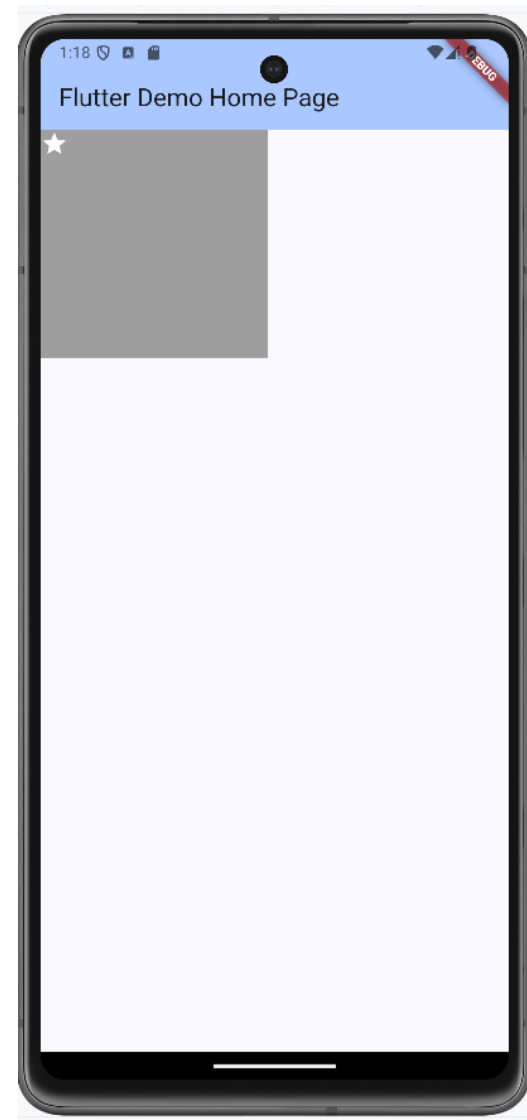
Align

- Align define a posição do widget filho dentro do seu pai, utilizando um sistema de coordenadas cartesiano que vai de -1.0 a 1.0.



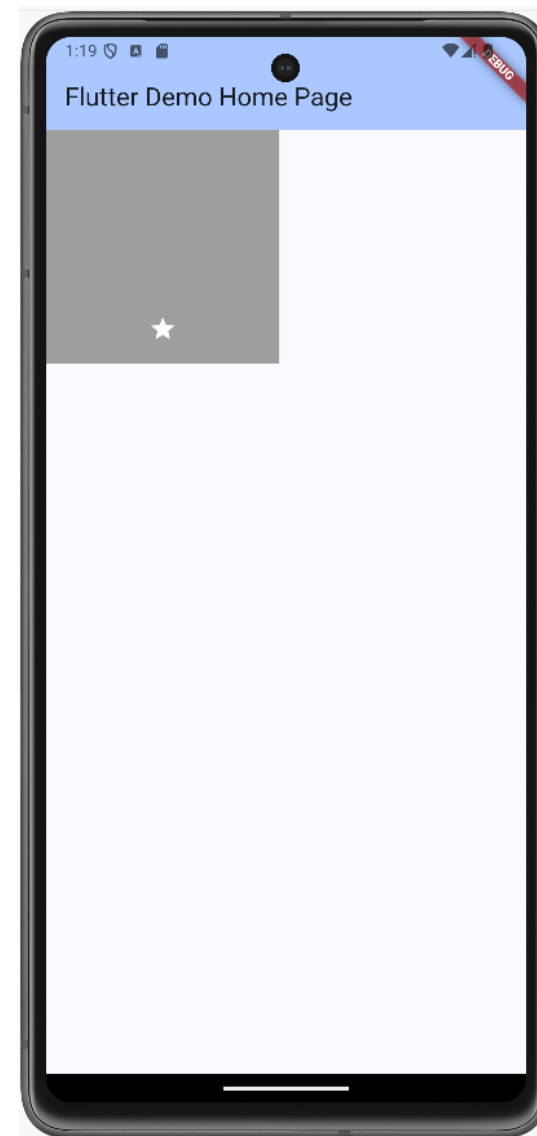
Align

```
Container(  
  width: 200,  
  height: 200,  
  color: Colors.grey,  
  child: Align(  
    alignment: Alignment.topLeft,  
    child: Icon(Icons.star, color: Colors.white),  
  ),  
)
```



Align

```
Container(  
  width: 200,  
  height: 200,  
  color: Colors.grey,  
  child: Align(  
    alignment: Alignment(0, 0.8),  
    child: Icon(Icons.star, color: Colors.white),  
  ),  
)
```



Expanded

- Expanded força o widget filho a **ocupar todo o espaço restante** disponível em uma Row ou Column.

Expanded

```
Column(  
  children: [  
    Row(  
      children: [  
        Expanded(  
          child: ColoredBox(  
            color: Colors.red,  
            child: Text('Item 1'),  
          ),  
        ),  
        Expanded(child: Text('Item 2')),  
      ],  
    ),  
    Row(  
      children: [  
        ColoredBox(  
          color: Colors.red,  
          child: Text('Item 3'),  
        ),  
        Text('Item 4'),  
      ],  
    ),  
  ],  
)
```

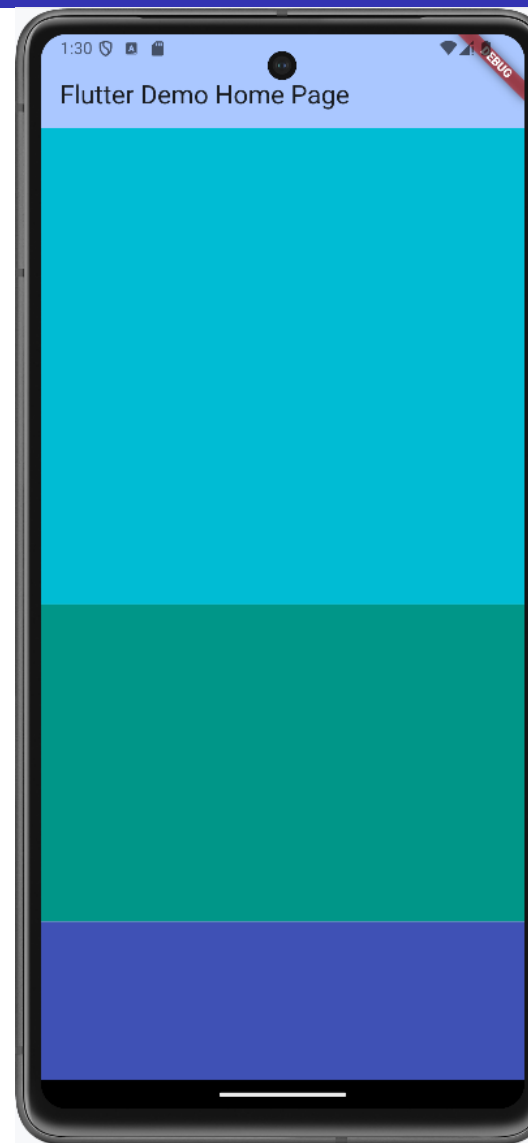


Flexible

- O Flexible oferece controle mais granular que o Expanded, permitindo que o widget filho ocupe o espaço disponível de forma opcional.

Flexible

```
Column(  
  children: [  
    Flexible(  
      flex: 3,  
      child: Container(  
        color: Colors.cyan,  
      ),  
    ),  
    Flexible(  
      flex: 2,  
      child: Container(  
        color: Colors.teal,  
      ),  
    ),  
    Flexible(  
      flex: 1,  
      child: Container(  
        color: Colors.indigo,  
      ),  
    ),  
  ],  
)
```



SizedBox

- O `SizedBox` é usado para definir dimensões exatas ou adicionar espaços vazios fixos entre widgets.

```
Column(  
  children: [  
    Text('Texto acima'),  
    SizedBox(height: 20),  
    Text('Texto abaixo com 20 pixels de distância'),  
  ],  
)
```



Stateless Widgets

- Os **Stateless Widgets** não possuem estado interno.
- O método "build()" é executado **uma única vez quando o widget é criado.**
- Os dados dentro de um Stateless Widgets **são imutáveis**, ou seja, não podem ser alterados a menos que uma nova instância do próprio widget seja inicializada novamente com diferentes configurações e propriedades.
 - **Dados vindos através de um construtor.**

Stateless Widgets

```
class Exemplo extends StatelessWidget {  
  const Exemplo({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Placeholder();  
  }  
}
```

```
class Exemplo extends StatelessWidget {  
  final String mensagem;  
  
  const Exemplo(String this.mensagem, {super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    print(mensagem);  
    return const Placeholder();  
  }  
}
```

Stateful Widgets

- Stateful widgets são **dinâmicos e carregam um estado interno mutável**.
- **Podem sofrer mudanças** ao longo da execução da aplicação sem a necessidade de reinicialização.
- Os Stateful widgets são usados quando precisamos de atualizações dinâmicas no estado do aplicativo em tempo de execução.
 - Contador, buscar dados de uma API, etc.
- Ao construir um Stateful widget o **Flutter cria um objeto de estado (state) vinculado ao widget**. O objeto de estado é responsável por manter todos os estados mutáveis para o widget.

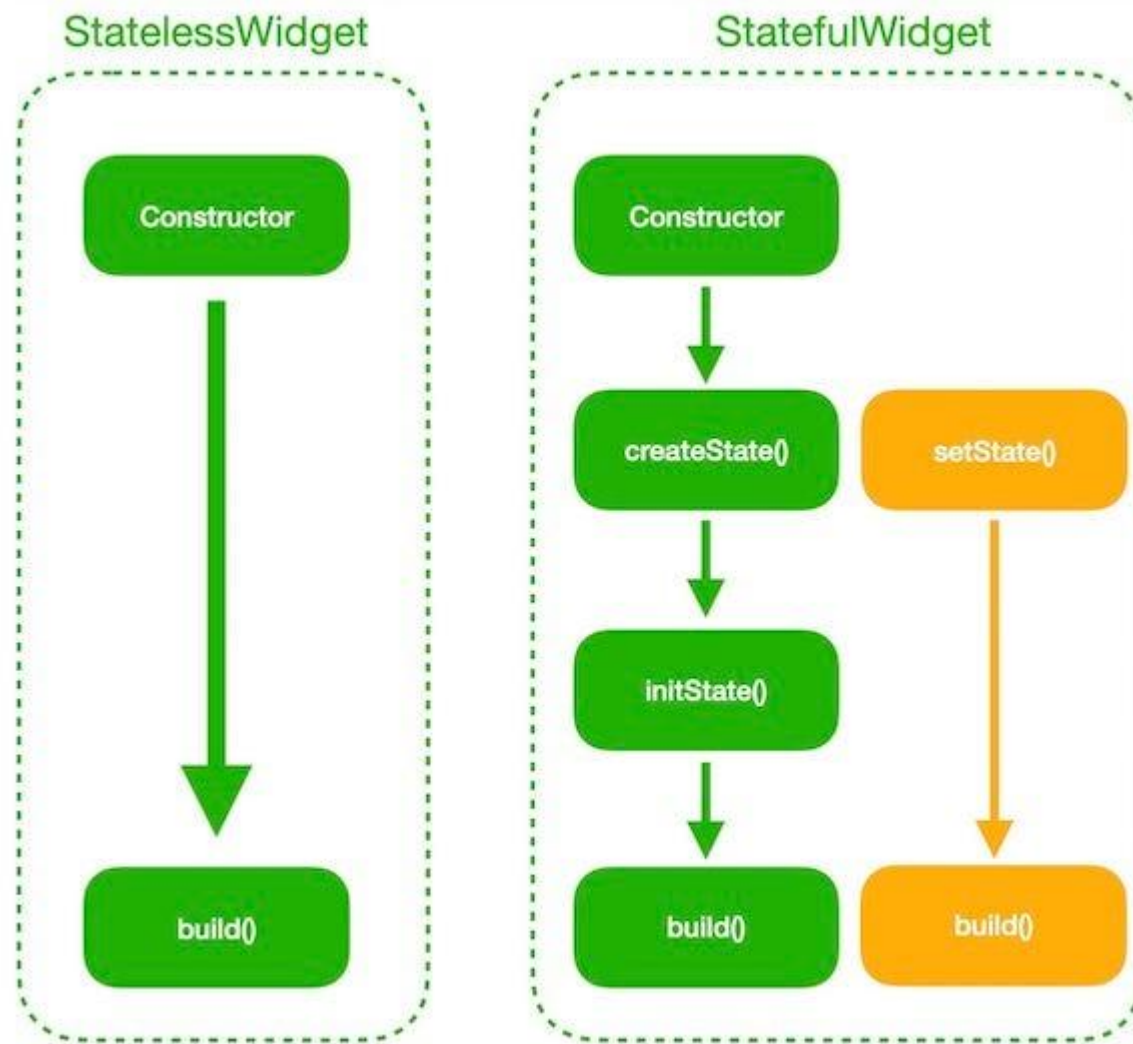
Stateful Widgets

- Os Stateful Widgets têm seu ciclo de vida baseado em seu estado e em como esse estado muda.
 - O construtor é invocado e em seguida o método "createState()".

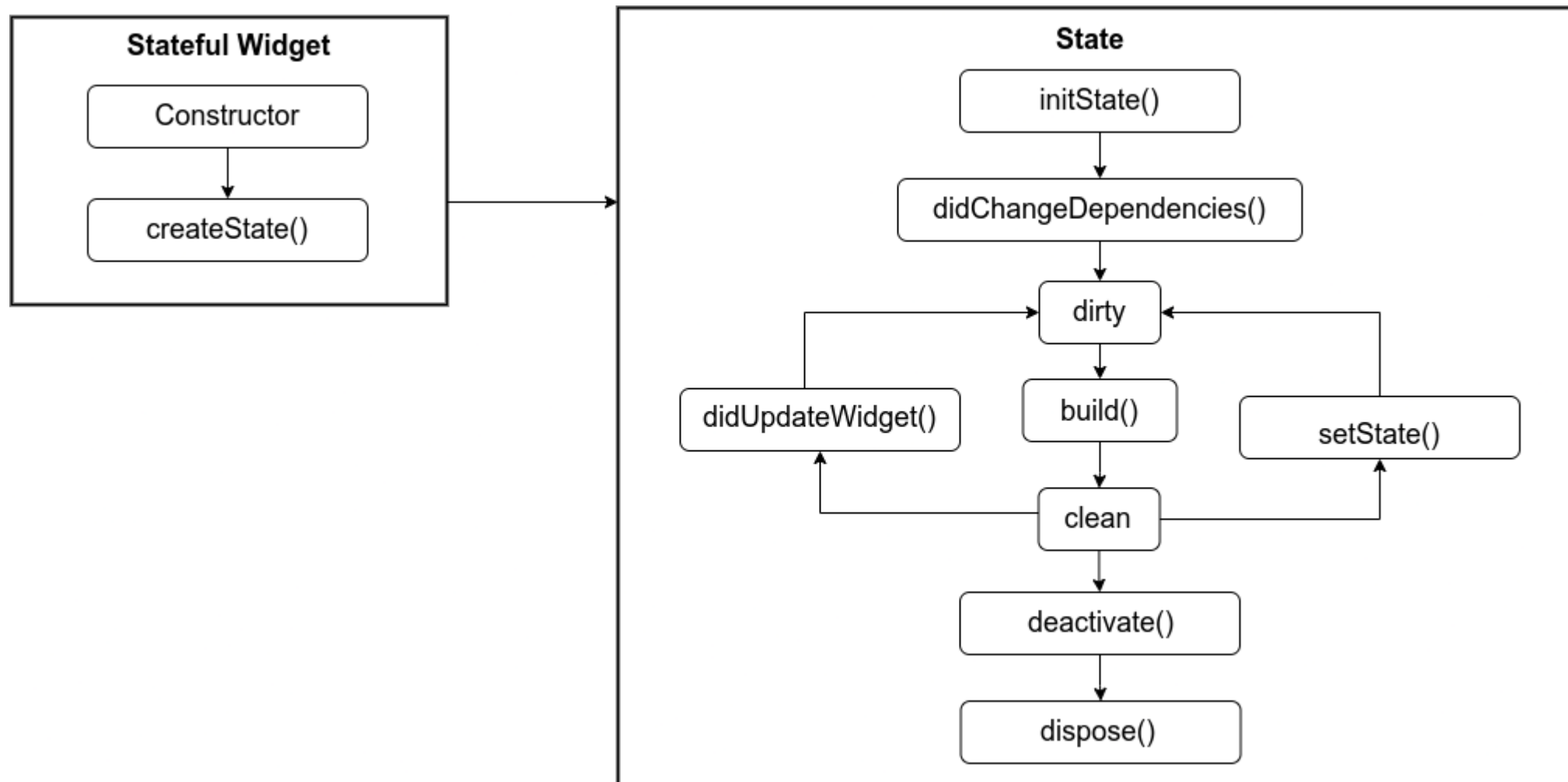
```
class Exemplo extends StatefulWidget {  
  const Exemplo({super.key});  
  
  @override  
  State<Exemplo> createState() => _ExemploState();  
}  
  
class _ExemploState extends State<Exemplo> {  
  @override  
  Widget build(BuildContext context) {  
    return const Placeholder();  
  }  
}
```

Ciclo de vida dos Widgets

- O ciclo de vida do widget é uma sequência de eventos que ocorrem quando um widget é criado, atualizado ou destruído.
- Compreender o ciclo de vida do widget é importante para escrever aplicativos Flutter eficientes.



Stateful Widgets Stateful em Detalhes



createState()

- Responsável por criar um objeto de estado (State).
- Este método deve ser implementado (`@override`)
- O objeto de estado **mantém todos os estados mutáveis**.
- Todos os widgets têm uma propriedade booleana chamada "mounted".

createState()

- Após a criação de um objeto de estado (State), o framework o associa a um **"BuildContext"** definindo **"mounted"** como **verdadeiro** (true).
- mounted verdadeiro indica que o objeto de estado e o widget associado estão na árvore de widgets.
- A propriedade "mounted" é útil quando um método de estado chama o "setState()", mas não está claro quantas vezes o método em questão é chamado.
 - Chamar o "setState()" quando um widget não está montado (unmounted) pode causar um erro.

createState()

```
Future<void> _navigateTelaPreenche(BuildContext context, String valor) async {  
  final result = await Navigator.push(  
    context,  
    MaterialPageRoute(builder: (context) => TelaPreenche(varName: valor)),  
  );  
  
  if (!mounted) return;  
  
  setState(() {  
    valorx = result;  
  });  
  
  ScaffoldMessenger.of(context)  
    ..removeCurrentSnackBar()  
    ..showSnackBar(  
      SnackBar(  
        content: Text('$result'),  
      ),  
    );  
}
```

Atenção

- Caso o objeto não esteja em estado mounted o setState não deve ser invocado.

initState()

- Uma vez que o objeto de estado (State) é inserido na árvore de widgets, o método "initState()" é executado automaticamente após o construtor da classe.
- Este método é estritamente **executado apenas uma vez**.
- Também é necessário chamar o método "super.initState()".
 - O "super.initState()" encaminha para a implementação padrão da classe base "State<T>" do seu widget, o que ajuda os widgets a funcionar corretamente.
- Dentro do "initState()", podemos inicializar variáveis, dados, propriedades, etc., e nos inscrever em Streams, ChangeNotifiers ou **qualquer outro objeto que possa alterar os dados em um widget específico**.

didChangeDependencies()

- É chamado **quando há uma mudança de dependência** em um objeto de estado que um determinado widget depende.
- Também é chamado imediatamente após o "initState()".
- Subclasses raramente usam esse método, pois o framework sempre aciona o "build()" após uma mudança de dependência.
- Este método é mais usado por subclasses em casos em que buscas de rede precisam ser realizadas após uma mudança de dependência, **o que de outra forma seria muito caro de fazer a cada construção ("build")**.

build()

- É responsável por **descrever e renderizar widgets** e suas mudanças na tela, constituindo a interface do usuário de qualquer aplicativo.
- Ao ser executado, ele **retorna uma nova árvore de widgets** que são configurados com informações provenientes do construtor deste widget e do "BuildContext" fornecido.

didUpdateWidget()

- Se um widget pai atualiza ou altera suas configurações e requer a reconstrução de widgets com o mesmo "runtimeType" e "Widget.key", o método "didUpdateWidget()" é chamado.
- Ele recebe o widget antigo como argumento, que pode ser usado para comparar com o novo widget.

setState()

- É um método especial para **notificar o Flutter sobre estados sujos**, ou seja, mudanças no objeto de estado e a necessidade de reconstruir o widget necessário
- Pode ser automaticamente invocado pelo framework e também pode ser chamado pelos desenvolvedores.
- Ao chamar o "setState()", a função "build" é acionada para o objeto de estado correspondente, que por sua vez atualiza a interface do usuário.
- Portanto, é importante entender que a mudança no estado de um objeto de estado deve ser feita dentro do método "setState()".

deactivate()

- É chamado quando o estado é temporariamente removido da árvore de widgets.

dispose()

- É chamado para remover permanentemente um objeto de estado de uma árvore de widgets.
- Com isso, a propriedade "mounted" do objeto de estado é definida como "false", indicando que ele nunca será construído novamente.
- **Dentro do método "dispose()", liberamos os recursos mantidos pelo objeto correspondente.**

dispose()

```
class _StateTelaCalculadora extends State<TelaCalculadora> {  
  TextEditingController controller = TextEditingController();  
  
  @override  
  void dispose(){  
    super.dispose();  
    controller.dispose();  
  }  
}
```

Atenção

- Uso do dispose para liberar o controller instanciado.

Referências

- <https://dart.dev/>
- <https://dart-tutorial.com/>
- <https://docs.flutter.dev/ui>
- <https://docs.flutter.dev/ui/widgets>
- <https://docs.flutter.dev/ui/layout>
- <https://docs.flutter.dev/resources/architectural-overview>